PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
Graduação em Engenharia de Computação - Disciplina de Sistemas Reconfiguráveis

RODRIGO PRETES MAIA
ISAAC ALVES SCHUENCK

DESCRIÇÃO DE CIRCUITOS LÓGICOS SEQUENCIAIS UTILIZANDO VHDL

Belo Horizonte
03 de dezembro de 2023

RODRIGO PRETES MAIA
ISAAC ALVES SCHUENCK

DESCRIÇÃO DE CIRCUITOS LÓGICOS SEQUENCIAIS UTILIZANDO VHDL

Trabalho desenvolvido como parte dos projetos
da disciplina de Sistemas Reconfiguráveis

Professor: Antônio Hamilton Magalhaes

Belo Horizonte 2023

Lista de Figuras

Figura 1: Diagrama equivalente do Port_io.
Figura 2: Bibliotecas e Entidade Port_io.
Figura 3: Arquitetura 1 Port_io.
Figura 4: Arquitetura 2 – Operação de escrita nos registradores.
Figura 5: Arquitetura 3 – Operação de leitura nos registradores.
Figura 6: Arquitetura 4 – Lógica da porta.
Figura 7: Arquitetura 5 – Lógica do latch e encerramento da arquitetura.
Figura 8: Simulação 1.
Figura 9: Simulação 2
Figura 10: Diagrama equivalente do RAM_mem.
Figura 11: Bibliotecas e Entidade RAM_mem.
Figura 12: Arquitetura 1 RAM_mem.
Figura 13: Arquitetura 2 RAM_mem.
Figura 14: Simulação RAM_mem.

Sumário

I. INTRODUÇÃO 5
II. PORT_IO 5
II.I - Entradas e Saídas do lado do processador 5
II.II - Lado da porta 5
II.III - Edereçamento 5
II.IV - Funcionamento 6
II.II - Diagrama equivalente 6
II.V - Descrição Port_io em VHDL 6
II.VI - Simulação e teste no Quartus II 8
III. RAM_MEM 9
III.I - Entradas e Saídas 9
III.II - Endereçamento 10
III.II - Diagrama equivalente 10
III.III - Descrição RAM_mem em VHDL 11
III.IV - Simulação e teste no Quartus II 13
IV. CONCLUSÃO 13
V. REFERENCIAS 13

I. INTRODUÇÃO

Este trabalho propõe a descrição, implementação em linguagem VHDL, simulação e análise de resultados de dois blocos independentes. Cada bloco representa um projeto distinto, desenvolvido em pastas separadas para organização. Utilizando a versão 9.1sp2 do software Quartus, o objetivo é criar códigos completos em VHDL, empregando tanto lógica concorrente quanto sequencial, com todas as entradas e saídas do tipo STD_LOGIC ou STD_LOGIC_VECTOR.

II. Port_io

O bloco port_io apresenta uma dualidade funcional distinta. Um de seus aspectos engloba as entradas e saídas destinadas à interação com o barramento interno do processador, enquanto o outro lado contempla interfaces de entrada ou saída destinadas à interação externa ao processador. A direção dessas conexões externas é determinada de maneira flexível por meio de um registrador, conferindo adaptabilidade e controle ao sistema.

II.I - Entradas e Saídas do lado do processador

Entradas:

- nrst: Reset assíncrono; zera port_reg e define tris_reg como '1'.
- clk_in: Clock do sistema para operações de escrita síncronas.
- abus_in[8..0]: Endereçamento para registradores internos.
- dbus_in[7..0]: Barramento de entrada para escrita nos registradores.
- wr_en: Habilitação para escrita; sincronizado com clk_in.
- rd_en: Habilitação para leitura; dbus_out recebe conteúdo de tris_reg ou latch de entrada.

Saída:

- dbus_out[7..0]: Barramento de saída para leitura; alta impedância quando não em uso.

II.II - Lado da porta

- port_io[7..0]: Porta bidirecional de 8 bits (INOUT). A direção é configurada bit a bit por meio do registrador tris_reg.

II.III - Endereçamento

- port_addr: Endereço para escrita no registrador port_reg (quando wr_en = ‘1’) ou leitura do latch (quando rd_en = ‘1’).
- tris_addr: Endereço para escrita no registrador tris_reg (quando wr_en = ‘1’) ou leitura do mesmo registrador (quando rd_en = ‘1’).
- alt_port_addr: Endereço alternativo para port_addr. Configurado com o mesmo valor de port_addr quando não em uso.
- alt_tris_addr: Endereço alternativo para tris_addr. Configurado com o mesmo valor de tris_addr quando não em uso.

II.IV - Funcionamento

Existem dois registradores neste bloco. O tris_reg configura a função de cada bit da porta como entrada ou saída, sendo escrita na borda de subida de clk_in. O port_reg armazena os dados de saída da porta, sendo escrito quando abus_in = port_addr ou abus_in = alt_port_addr e wr_en = '1'. Após o reset, todos os bits ficam zerados. Se um bit da porta está configurado como saída, seu valor é determinado pelo bit correspondente em port_reg; caso seja configurado como entrada, o valor em port_reg não tem efeito. A leitura ocorre nos mesmos endereços de escrita em port_reg, lendo um latch que trava os valores presentes na porta. Normalmente, esse latch está no estado transparente.

II.II - Diagrama equivalente

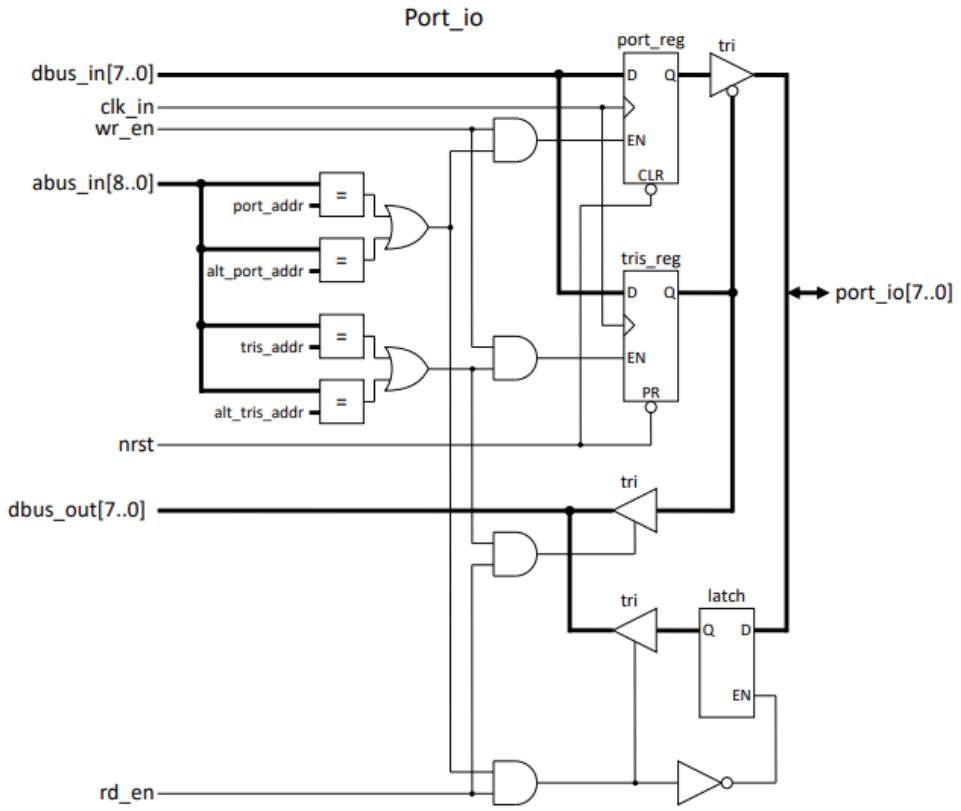

Figura 1: Diagrama equivalente do Port_io.

II.V - Descrição Port_io em VHDL

Primeiro houve a declaração das bibliotecas e das entidades que vamos utilizar.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
use ieee.numeric_std.all;

entity Port_io is
    generic (
        port_addr : STD_LOGIC_VECTOR (8 downto 0) := "000000000";
        tris_addr : STD_LOGIC_VECTOR (8 downto 0) := "000000001";
        alt_port_addr : STD_LOGIC_VECTOR (8 downto 0) := "000000010";
        alt_tris_addr : STD_LOGIC_VECTOR (8 downto 0) := "000000011"
    );
    port (
        nrst : in STD_LOGIC;
        clk_in : in STD_LOGIC;
        abus_in : in STD_LOGIC_VECTOR (8 downto 0);
        dbus_in : in STD_LOGIC_VECTOR (7 downto 0);
        wr_en : in STD_LOGIC;
        rd_en : in STD_LOGIC;
        dbus_out : out STD_LOGIC_VECTOR (7 downto 0);
        port_io : inout STD_LOGIC_VECTOR (7 downto 0)
    );
end Port_io;
```

Figura 2: Bibliotecas e Entidade Port_io.

Após a declaração das entidades definimos a arquitetura.

```
architecture Port_IO of Port_io is
    signal tris_reg : STD_LOGIC_VECTOR (7 downto 0) := (others => '1');
    signal port_reg : STD_LOGIC_VECTOR (7 downto 0) := (others => '0');
    signal latch : STD_LOGIC_VECTOR (7 downto 0) := (others => '0');
```

Figura 3: Arquitetura 1 Port_io.

```
    -- operação de escrita nos registradores
    process (nrst, clk_in)
    begin
        if nrst = '0' then
            tris_reg <= (others => '1'); -- Todos os bits setados para '1'
            port_reg <= (others => '0'); -- Todos os bits setados para '0'
        elsif rising_edge(clk_in) then
            if wr_en = '1' then
                if abus_in = tris_addr or abus_in = alt_tris_addr then
                    tris_reg <= dbus_in(7 downto 0);
                elsif abus_in = port_addr or abus_in = alt_port_addr then
                    port_reg <= dbus_in(7 downto 0);
                end if;
            end if;
        end if;
    end process;
```

Figura 4: Arquitetura 2 – Operação de escrita nos registradores.

```
-- operação de leitura dos registradores
process (rd_en, abus_in, latch, tris_reg)
begin
    if rd_en = '1' then
        if abus_in = port_addr or abus_in = alt_port_addr then
            dbus_out <= latch;
        elsif abus_in = tris_addr OR abus_in = alt_tris_addr THEN
            dbus_out <= tris_reg;
        else
            dbus_out <= (OTHERS => 'Z');
        end if;
    else
        dbus_out <= (OTHERS => 'Z'); -- Missing semicolon here
    end if;
end process;
```

Figura 5: Arquitetura 3 – Operação de leitura nos registradores.

```
-- lógica da porta
process (tris_reg, port_reg, port_io)
begin
    for i in 0 to 7 loop
        -- se algum bit de tris_reg for saida, port_io recebe o bit dessa posicao de port_reg
        if tris_reg(i) = '0' then
            port_io(i) <= port_reg(i);
            -- caso não seja a posicao fica em alta impedancia
        else
            port_io(i) <= 'Z';
        end if;
    end loop;
end process;
```

Figura 6: Arquitetura 4 – Lógica da porta.

```
-- lógica do latch
process (rd_en, abus_in, port_io)
begin
    if rd_en = '1' then
        if abus_in = port_addr or abus_in = alt_port_addr then
            latch <= port_io;
        end if;
    end if;
end process;

end Port_IO;
```

Figura 7: Arquitetura 5 – Lógica do latch e encerramento da arquitetura.

II.VI - Simulação e teste no Quartus II

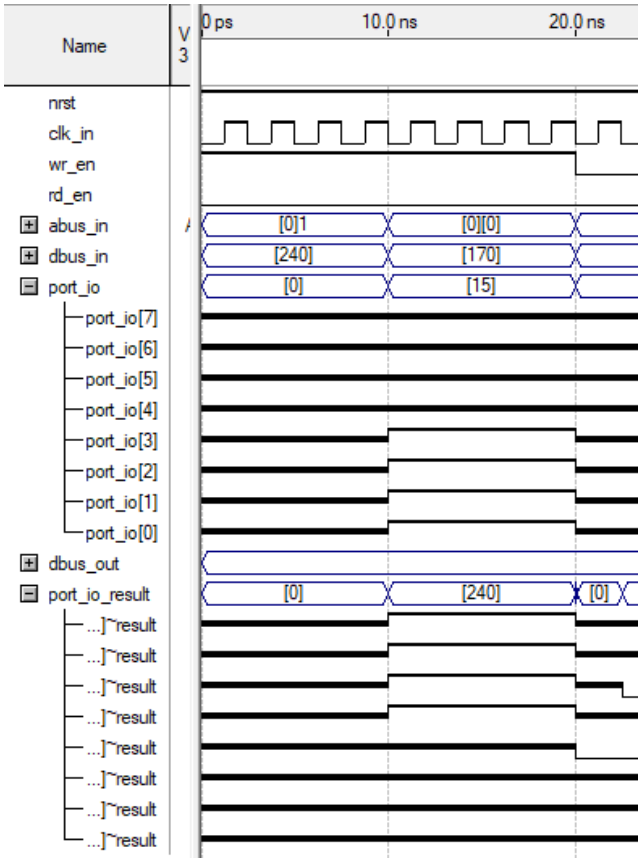

Figura 8: Simulação 1.

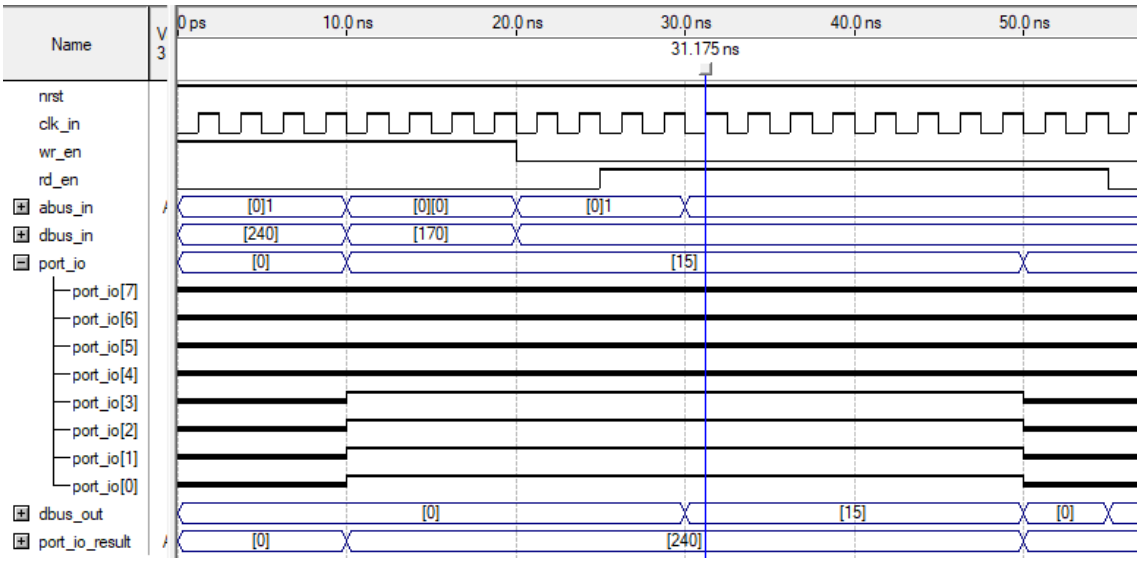

Figura 9: Simulação 2

III. RAM_mem

O bloco RAM_mem engloba quatro áreas de memória denominadas registros de propósito geral, consistindo em três áreas de 80 bytes e uma com 16 bytes.

III.I - Entradas e Saídas

Entradas:

- nrst: Reset assíncrono (nível lógico baixo) zera todos os bits da RAM, com prioridade.

- clk_in: Clock do sistema para escrita na borda de subida, quando habilitado.
- abus_in[8..0]: Endereçamento para escrita/leitura na memória, seguindo faixas especificadas.
- dbus_in[7..0]: Entrada de dados para escrita, habilitada por wr_en, endereçada por abus_in.
- wr_en: Habilitação para escrita síncrona na memória com clk_in, se endereço presente em abus_in.
- rd_en: Habilitação para leitura, dbus_out corresponde ao conteúdo da memória se endereço presente.

Saídas:

- dbus_out[7..0]: Barramento de saída de dados (8 bits), habilitado por rd_en e endereçado por abus_in. Comportamento combinacional durante leitura, alta impedância quando não em uso.

III.II - Endereçamento

- mem0 20h ~ 6Fh (80 bytes). Em decimal: 32 a 111
- mem1 A0h ~EFh (80 bytes). Em decimal: 160 a 239
- mem2 120h ~16Fh (80 bytes). Em decimal: 288 a 367
- mem_com 70h ~ 7Fh (16 bytes). Em decimal: 112 a 127

III.II - Diagrama equivalente

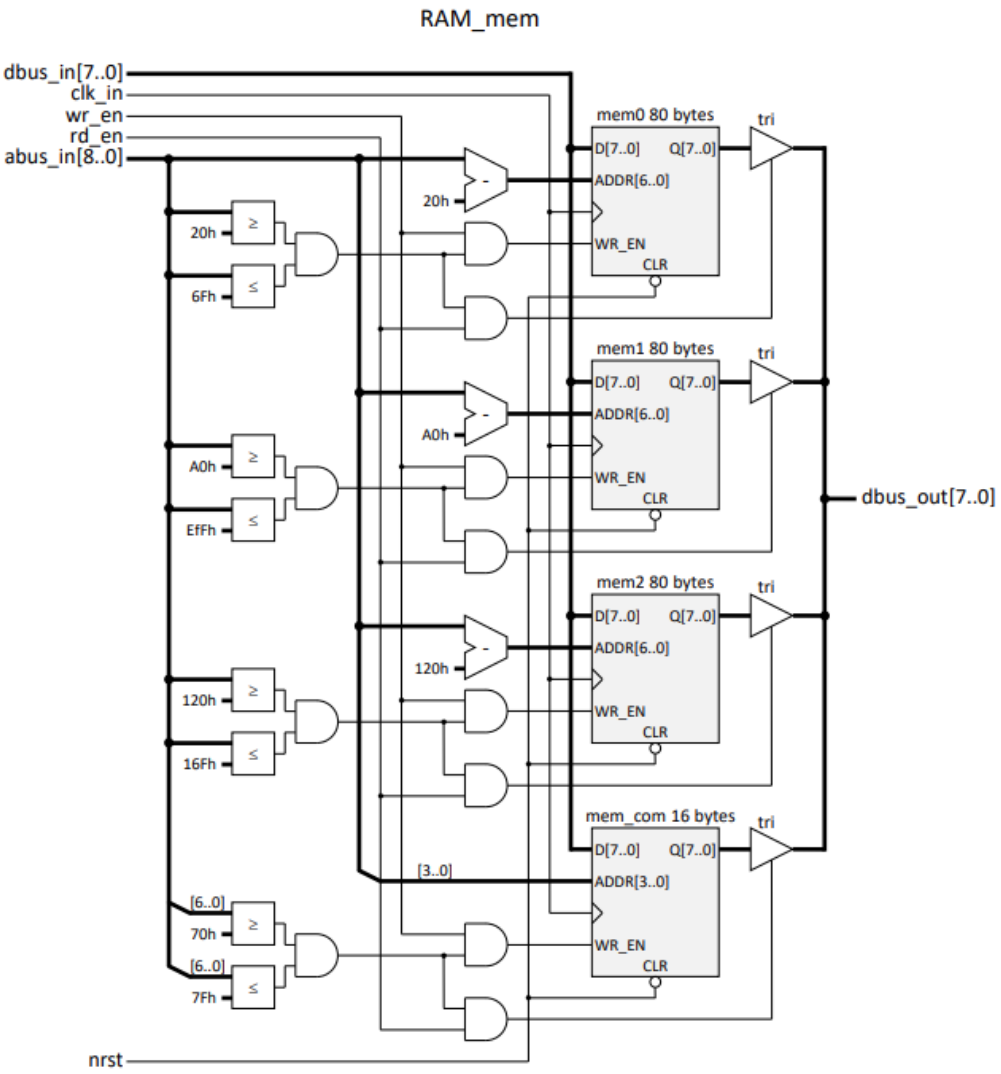

Figura 10: Diagrama equivalente do RAM_mem.

III.III - Descrição RAM_mem em VHDL

Primeiro houve a declaração das bibliotecas e das entidades que vamos utilizar.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
--use IEEE.STD_LOGIC_ARITH.ALL;
--use IEEE.STD_LOGIC_UNSIGNED.ALL;
USE IEEE.NUMERIC_STD.ALL;

entity RAM_mem is
    Port ( nrst    : in STD_LOGIC;
           clk_in  : in STD_LOGIC;
           abus_in : in STD_LOGIC_VECTOR(8 downto 0);
           dbus_in : in STD_LOGIC_VECTOR(7 downto 0);
           wr_en   : in STD_LOGIC;
           rd_en   : in STD_LOGIC;
           dbus_out : out STD_LOGIC_VECTOR(7 downto 0));
           --addr : STD_LOGIC_VECTOR(4 DOWNTO 0);
end RAM_mem;
```

Figura 11: Bibliotecas e Entidade RAM_mem.

Após a declaração das entidades definimos a arquitetura.

```
architecture ram of RAM_mem is
    type memory_array is array (natural range <>) of STD_LOGIC_VECTOR(7 downto 0);
    signal mem0 : memory_array(32 to 111);
    signal mem1 : memory_array(160 to 239);
    signal mem2 : memory_array(288 to 367);
    signal mem_com : memory_array(112 to 127);
    SIGNAL abus_int : INTEGER RANGE 0 TO 367;
```

Figura 12: Arquitetura 1 RAM_mem.

```
begin
    process (nrst, clk_in)
    begin
    --conversao
    abus_int <= TO_INTEGER(UNSIGNED(abus_in));
        if nrst = '0' then
            -- Reset
            mem0 <= (others => (others => '0'));
            mem1 <= (others => (others => '0'));
            mem2 <= (others => (others => '0'));
            mem_com <= (others => (others => '0'));
        elsif rising_edge(clk_in) then
            -- Escrita
            if wr_en = '1' then
                case abus_int is
                    when 32 to 111 =>
                        mem0(abus_int) <= dbus_in;
                    when 160 to 239 =>
                        mem1(abus_int) <= dbus_in;
                    when 288 to 367 =>
                        mem2(abus_int) <= dbus_in;
                    when 112 to 127 =>
                        mem_com(abus_int) <= dbus_in;
                    when others =>

                end case;
            end if;
            -- Leitura
            if rd_en = '1' then
                case abus_int is
                    when 32 to 111 =>
                        dbus_out <= mem0(abus_int);
                    when 160 to 239 =>
                        dbus_out <= mem1(abus_int);
                    when 288 to 367 =>
                        dbus_out <= mem2(abus_int);
                    when 112 to 127 =>
                        dbus_out <= mem_com(abus_int);
                    when others =>
                end case;
            else
                dbus_out <= (others => 'Z');
            end if;
        end if;
    end process;
end ram;
```

Figura 13: Arquitetura 2 RAM_mem.

III.IV - Simulação e teste no Quartus II

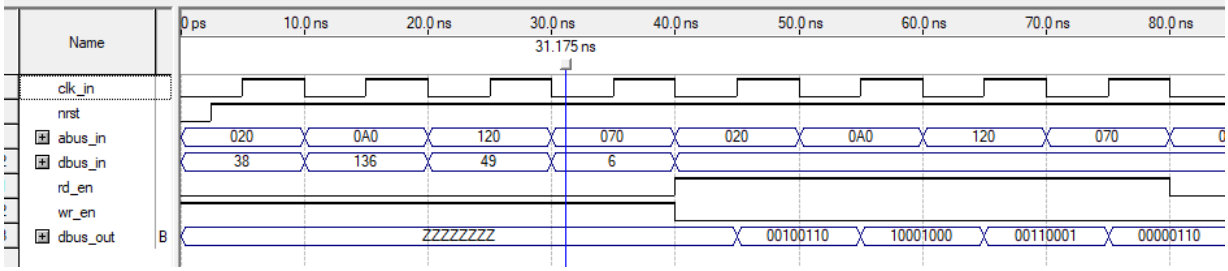

Figura 14: Simulação RAM_mem.

IV. CONCLUSÃO

Ao longo deste trabalho, exploramos a intricada tarefa de descrever, implementar, simular e analisar dois blocos independentes por meio da linguagem VHDL. A decisão de organizar cada projeto em pastas separadas refletiu a busca pela máxima clareza e organização durante o desenvolvimento.

A escolha de utilizar exclusivamente entradas e saídas do tipo STD_LOGIC ou STD_LOGIC_VECTOR contribuiu para a padronização e coesão do código, facilitando a compreensão e manutenção futura.

Através da simulação e análise de resultados, validamos a eficácia dos blocos independentes, destacando a robustez e confiabilidade das soluções propostas. Este trabalho não apenas aprimorou nossas habilidades em VHDL, mas também proporcionou uma compreensão mais profunda dos princípios de design digital, preparando-nos para desafios mais complexos no campo da engenharia eletrônica.

V. REFERENCIAS

Material disponível na disciplina de Sistemas Reconfiguráveis. Canvas Puc Minas, 2023. Disponível em: https://pucminas.instructure.com/courses/161258. Acesso em: 03 out. 2023.